

COMP90042 NLP

L6: Hidden Markov Models



Jey Han Lau

POS Tagging Recap

- Janet will back the bill
- Janet/**NNP** will/**MB** back/**VB** the/**DT** bill/**NN**
- Local classifier: prone to **error propagation**
- What about treating the full sequence as a “class”?
 - Output: “NNP_MB_VB_DT_NN”
- Problems:
 - Exponentially many combinations: $|Tags|^M$, for length M
 - How to tag sequences of different lengths?

A Better Approach

- Tagging is a sentence-level task but as humans we **decompose** it into small word-level tasks.
 - Janet/**NNP** will/**MD** back/**VB** the/**DT** bill/**NN**
- Solution:
 - Define a model that decomposes process into individual word level steps
 - But that takes into account the whole sequence when learning and predicting (no error propagation)
- This is the idea of **sequence labelling**, and more general, **structured prediction**.

A Probabilistic Model

- Goal: obtain best tag sequence $\mathbf{t}$ from sentence $\mathbf{w}$
 - $\hat{\mathbf{t}} = \operatorname{argmax}_{\mathbf{t}} P(\mathbf{t} \mid \mathbf{w})$
 - $\hat{\mathbf{t}} = \operatorname{argmax}_{\mathbf{t}} \frac{P(\mathbf{w} \mid \mathbf{t}) P(\mathbf{t})}{P(\mathbf{w})} = \operatorname{argmax}_{\mathbf{t}} P(\mathbf{w} \mid \mathbf{t}) P(\mathbf{t})$ [Bayes]
- Let's decompose:
 - $P(\mathbf{w} \mid \mathbf{t}) = \prod_{i=1}^n P(w_i \mid t_i)$ [Prob. of a word depends only on the tag]
 - $P(\mathbf{t}) = \prod_{i=1}^n P(t_i \mid t_{i-1})$ [Prob. of a tag depends only on the previous tag]
- This is a Hidden Markov Model (HMM)

Two Assumptions of HMM

$$\hat{t} = \operatorname{argmax}_t P(\mathbf{w} | t) P(t)$$

$$P(\mathbf{w} | t) = \prod_{i=1}^n P(w_i | t_i)$$

$$P(t) = \prod_{i=1}^n P(t_i | t_{i-1})$$

- Output independence
 - An observed event (word) depends only on the hidden state (tag)
- Markov assumption
 - The current state (tag) depends only on previous state

HMMs - Training

- Parameters are the individual probabilities
 - $P(w_i | t_i) = \textbf{emission (O)}$ probabilities
 - $P(t_i | t_{i-1}) = \textbf{transition (A)}$ probabilities
- Training uses Maximum Likelihood Estimation (MLE)
 - This is done by simply counting word frequencies according to their tags (just like N -gram language models!)

$$\bullet \quad P(\textit{like} | VB) = \frac{\textit{count}(VB, \textit{like})}{\textit{count}(VB)}$$

$$\bullet \quad P(NN | DT) = \frac{\textit{count}(DT, NN)}{\textit{count}(DT)}$$

HMMs - Training

- What about the first tag?
 - Assume we have a symbol “<s>” that represents the starting state/tag

$$P(NN \mid < s >) = \frac{\text{count}(< s >, NN)}{\text{count}(< s >)}$$

- What about unseen (word, tag) and (tag, previous_tag) combinations?
 - Smoothing techniques

Transition Matrix

	NNP	MD	VB	JJ	NN	RB	DT
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

Figure 8.7 The A transition probabilities $P(t_i|t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(VB|MD)$ is 0.7968.

Emission (Observation) Matrix

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Figure 8.8 Observation likelihoods B computed from the WSJ corpus without smoothing, simplified slightly.

HMMs – Prediction (Decoding)

$$\begin{aligned}\hat{t} &= \operatorname{argmax}_t P(\mathbf{w} | t) P(t) \\ &= \operatorname{argmax}_t \prod_{i=1}^n P(w_i | t_i) P(t_i | t_{i-1})\end{aligned}$$

- Simple idea: for each word, take the tag that maximises $P(w_i | t_i) P(t_i | t_{i-1})$. Do it left-to-right, in *greedy* fashion.
- This is wrong! We are looking for argmax_t , not individual $\operatorname{argmax}_{t_i}$ terms.
 - This is a local classifier: error propagation
- Correct way: consider **all** possible tag combinations, evaluate them, take the max

The Viterbi Algorithm

- Dynamic Programming to the rescue!
 - We can still proceed sequentially, as long as we are careful.
- POS tag: “can play”
- Best tag for “can” is easy: $\operatorname{argmax}_t P(\text{can} \mid t) P(t \mid \langle s \rangle)$
- Suppose best tag for “can” is NN. To get the tag for “play”, can we take $\operatorname{argmax}_t P(\text{play} \mid t) P(t \mid \text{NN})$?
- No! Instead, we keep track of **scores for each tag** for “can” and check them with the different tags of “play”.

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP					
MD					
VB					
JJ					
NN					
RB					
DT					

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	$P(\text{Janet} \text{NNP}) * P(\text{NNP} \text{<s>})$				
MD					
VB					
JJ	...				
NN	...				
RB	...				
DT	...				

$$P(\text{Janet} \mid \text{NNP}) = ?; P(\text{NNP} \mid \langle s \rangle) = ?$$

	NNP	MD	VB	JJ	NN	RB	DT
$\langle s \rangle$	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

Figure 8.7 The A transition probabilities $P(t_i|t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(VB|MD)$ is 0.7968.

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Figure 8.8 Observation likelihoods B computed from the WSJ corpus without smoothing, simplified slightly.

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	$0.000032 * 0.2767 = 8.8544e-06$				
MD	$P(\text{Janet} \text{MD}) * P(\text{MD} \text{<s>})$				
VB	...				
JJ	...				
NN	...				
RB	...				
DT	...				

$$P(\text{Janet} \mid \text{MD}) = ?; P(\text{MD} \mid \langle s \rangle) = ?$$

	NNP	MD	VB	JJ	NN	RB	DT
$\langle s \rangle$	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

Figure 8.7 The A transition probabilities $P(t_i|t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(\text{VB}|\text{MD})$ is 0.7968.

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Figure 8.8 Observation likelihoods B computed from the WSJ corpus without smoothing, simplified slightly.

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●				
MD	0				
VB	0				
JJ	0				
NN	0				
RB	0				
DT	0				

$$P(\text{Janet} \mid \text{VB/JJ/etc}) = ?$$

	NNP	MD	VB	JJ	NN	RB	DT
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

Figure 8.7 The A transition probabilities $P(t_i|t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(\text{VB}|\text{MD})$ is 0.7968.

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Figure 8.8 Observation likelihoods B computed from the WSJ corpus without smoothing, simplified slightly.

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06	$P(\text{will} \text{NNP}) * P(\text{NNP} t_{\text{Janet}}) * s(t_{\text{Janet}} \text{Janet})$			
MD	0	...			
VB	0	...			
JJ	0	...			
NN	0	...			
RB	0	...			
DT	0	...			

Consider all possible tags, and take the maximum

$\max(P(\text{will} | \text{NNP}) * P(\text{NNP} | \text{NNP}) * s(\text{NNP} | \text{Janet}),$
 $P(\text{will} | \text{NNP}) * P(\text{NNP} | \text{MD}) * s(\text{MD} | \text{Janet}),$
...
 $P(\text{will} | \text{NNP}) * P(\text{NNP} | \text{DT}) * s(\text{DT} | \text{Janet}))$

$= P(\text{will} | \text{NNP}) * P(\text{NNP} | \text{NNP}) * 8.8544\text{e-}06$

$$P(\text{will} \mid \text{NNP}) = ?; P(\text{NNP} \mid \text{NNP}) = ?$$

	NNP	MD	VB	JJ	NN	RB	DT
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

Figure 8.7 The A transition probabilities $P(t_i|t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(VB|MD)$ is 0.7968.

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Figure 8.8 Observation likelihoods B computed from the WSJ corpus without smoothing, simplified slightly.

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0			
MD	0	...			
VB	0	...			
JJ	0	...			
NN	0	...			
RB	0	...			
DT	0	...			

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0			
MD	0	$P(\text{will} \text{MD}) * P(\text{MD} t_{\text{Janet}}) * s(t_{\text{Janet}} \text{Janet})$			
VB	0	...			
JJ	0	...			
NN	0	...			
RB	0	...			
DT	0	...			

$\max(P(\text{will} | \text{MD}) * P(\text{MD} | \text{NNP}) * s(\text{NNP} | \text{Janet}),$
 $P(\text{will} | \text{MD}) * P(\text{MD} | \text{MD}) * s(\text{MD} | \text{Janet}),$
...
 $P(\text{will} | \text{MD}) * P(\text{MD} | \text{DT}) * s(\text{DT} | \text{Janet}))$

$= P(\text{will} | \text{MD}) * P(\text{MD} | \text{NNP}) * 8.8544\text{e-}06$

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0			
MD	0	3.004e-08			
VB	0	...			
JJ	0	...			
NN	0	...			
RB	0	...			
DT	0	...			

$\max(P(\text{will} \mid \text{MD}) * P(\text{MD} \mid \text{NNP}) * s(\text{NNP} \mid \text{Janet}),$
 $P(\text{will} \mid \text{MD}) * P(\text{MD} \mid \text{MD}) * s(\text{MD} \mid \text{Janet}),$
...
 $P(\text{will} \mid \text{MD}) * P(\text{MD} \mid \text{DT}) * s(\text{DT} \mid \text{Janet}))$

$= P(\text{will} \mid \text{MD}) * P(\text{MD} \mid \text{NNP}) * 8.8544\text{e-}06$

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0			
MD	0	3.004e-8			
VB	0	2.231e-13			
JJ	0	0			
NN	0	1.034e-10			
RB	0	0			
DT	0	0			

Transition and Emission Matrix

	NNP	MD	VB	JJ	NN	RB	DT
<s>	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

Figure 8.7 The A transition probabilities $P(t_i|t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(VB|MD)$ is 0.7968.

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Figure 8.8 Observation likelihoods B computed from the WSJ corpus without smoothing, simplified slightly.

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0			
MD	0	3.004e-8			
VB	0	2.231e-13			
JJ	0	0			
NN	0	1.034e-10			
RB	0	0			
DT	0	0			

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0	0		
MD	0	3.004e-8	0		
VB	0	2.231e-13	$P(\text{back} \text{VB}) * P(\text{VB} t_{\text{will}}) * s(t_{\text{will}} \text{will})$		
JJ	0	0			
NN	0	1.034e-10			
RB	0	0			
DT	0	0			

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0	0		
MD	0	3.004e-8	0		
VB	0	2.231e-13	MD: 1.6e-11 VB: 7.5e-19 NN: 9.7e-17		
JJ	0	0			
NN	0	1.034e-10			
RB	0	0			
DT	0	0			

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0	0		
MD	0	3.004e-8	0		
VB	0	2.231e-13	MD: 1.6e-11 VB: 7.5e-19 NN: 9.7e-17		
JJ	0	0			
NN	0	1.034e-10			
RB	0	0			
DT	0	0			

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0	0		
MD	0	3.004e-8	0		
VB	0	2.231e-13	1.6e-11		
JJ	0	0			
NN	0	1.034e-10			
RB	0	0			
DT	0	0			

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0	0		
MD	0	3.004e-8	0		
VB	0	2.231e-13	1.6e-11		
JJ	0	0	5.1e-15		
NN	0	1.034e-10	5.4e-15		
RB	0	0	5.3e-11		
DT	0	0	0		

The Viterbi Algorithm

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0	0	2.5e-17	
MD	0	3.004e-8	0	0	
VB	0	2.231e-13	1.6e-11	0	
JJ	0	0	5.1e-15	0	
NN	0	1.034e-10	5.4e-15	0	
RB	0	0	5.3e-11	0	
DT	0	0	0	1.8e-12	

The diagram illustrates the Viterbi Algorithm using a grid of words and their corresponding probabilities. The words are arranged in columns: Janet, will, back, the, bill. The rows represent different parts of speech (NNP, MD, VB, JJ, NN, RB, DT). The probabilities are shown in the cells. Red dashed arrows indicate the backtracking path from the final state (DT, 1.8e-12) to the initial state (NNP, 8.8544e-06). The path starts at DT (1.8e-12), goes to RB (5.3e-11), then to VB (1.6e-11), then to MD (3.004e-8), and finally to NNP (8.8544e-06). A red dot is placed on the NNP cell.

Done!

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0	0	2.5e-17	0
MD	0	3.004e-8	0	0	0
VB	0	2.231e-13	1.6e-11	0	1.0e-20
JJ	0	0	5.1e-15	0	0
NN	0	1.034e-10	5.4e-15	0	2.0e-15
RB	0	0	5.3e-11	0	0
DT	0	0	0	1.8e-12	0

Going Backwards To Get The Best Tag Sequence

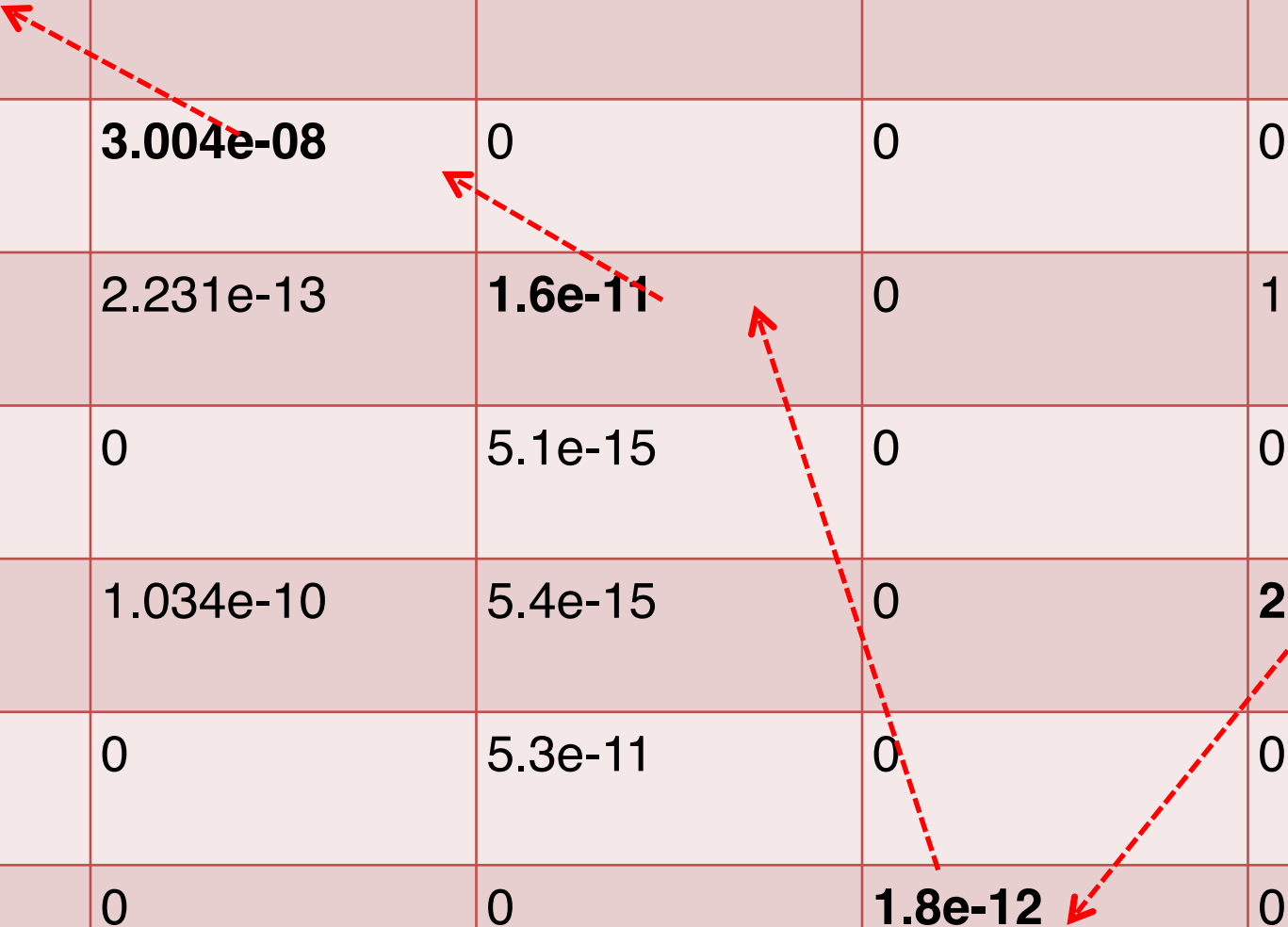
	Janet	will	back	the	bill
NNP	8.8544e-06	0	0	2.5e-17	0
MD	0	3.004e-8	0	0	0
VB	0	2.231e-13	1.6e-11	0	1.0e-20
JJ	0	0	5.1e-15	0	0
NN	0	1.034e-10	5.4e-15	0	2.0e-15
RB	0	0	5.3e-11	0	0
DT	0	0	0	1.8e-12	0

Going Backwards To Get The Best Tag Sequence

	Janet	will	back	the	bill
NNP	8.8544e-06 ●	0	0	2.5e-17	0
MD	0	3.004e-08	0	0	0
VB	0	2.231e-13	1.6e-11	0	1.0e-20
JJ	0	0	5.1e-15	0	0
NN	0	1.034e-10	5.4e-15	0	2.0e-15
RB	0	0	5.3e-11	0	0
DT	0	0	0	1.8e-12	0

Final Tagging Results

	Janet/ NNP	will/ MD	back/ VB	the/ DT	bill/ NN
NNP	8.8544e-06 ●	0	0	2.5e-17	0
MD	0	3.004e-08	0	0	0
VB	0	2.231e-13	1.6e-11	0	1.0e-20
JJ	0	0	5.1e-15	0	0
NN	0	1.034e-10	5.4e-15	0	2.0e-15
RB	0	0	5.3e-11	0	0
DT	0	0	0	1.8e-12	0



What is the optimal POS tag sequence for: THEY FISH

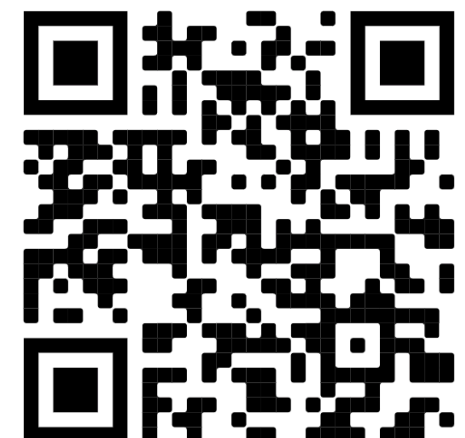
	they	fish
N	0.7	0.3
V	0.4	0.6

Word Emission Probabilities

	N	V
<S>	0.6	0.4
N	0.8	0.2
V	0.7	0.3

Transition Probabilities

[PollEv.com/jeyhanlau569](https://pollev.com/jeyhanlau569)



THEY FISH

	they	fish
N		
V		

PolEv.com/jeyhanlau569



The Viterbi Algorithm

- Complexity: $O(T^2N)$, where T is the size of the tagset and N is the length of the sequence.
 - $T * N$ matrix, each cell performs T operations.
- Why does it work?
 - Because of the **independence assumptions** that decompose the problem.
 - Without these, we cannot apply DP.

Viterbi Pseudocode

```
alpha = np.zeros(M, T)
for t in range(T):
    alpha[1, t] = pi[t] * O[w[1], t]

for i in range(2, M):
    for t_i in range(T):
        for t_last in range(T):           # t_last means t_{i-1}
            s = alpha[i-1, t_last] * A[t_last, t_i] * O[w[i], t_i]
            if s > alpha[i, t_i]:
                alpha[i, t_i] = s
                back[i, t_i] = t_last

best = np.max(alpha[M-1, :])
return backtrace(best, back)
```

- Good practice: work with **log** probabilities to prevent underflow (multiplications become sums)
- Vectorisation (use matrix-vector operations)

HMMs In Practice

- We saw HMM taggers based on **bigrams** (first order HMM).
 - I.e. current tag depends only the immediate previous tag
- State-of-the-art use tag **trigrams** (second order HMM).
 - $P(t) = \prod_{i=1}^n P(t_i | t_{i-1}, t_{i-2})$ Viterbi now $O(T^3N)$
- Need to deal with sparsity: some tag trigram sequences might not be present in training data
 - Interpolation: $P(t_i | t_{i-1}, t_{i-2}) = \lambda_3 \hat{P}(t_i | t_{i-1}, t_{i-2}) + \lambda_2 \hat{P}(t_i | t_{i-1}) + \lambda_1 \hat{P}(t_i)$
 - $\lambda_1 + \lambda_2 + \lambda_3 = 1$
- With additional features, reach 96.5% accuracy on Penn Treebank (Brants, 2000)

Generative vs. Discriminative Taggers

- HMM is **generative**

- $\hat{T} = \operatorname{argmax}_T P(T | W)$

- $= \operatorname{argmax}_T P(W | T) P(T)$

- $= \operatorname{argmax}_T \prod_i P(w_i | t_i) P(t_i | t_{i-1})$

- trained HMM can generate data (sentences)!
- allows for unsupervised HMMs: learn model without any tagged data!

Generative vs. Discriminative Taggers

$$\begin{aligned}\hat{T} &= \operatorname{argmax}_T P(T | W) \\ &= \operatorname{argmax}_T \prod_i P(t_i | w_i, t_{i-1})\end{aligned}$$

- **Discriminative** models describe $P(T | W)$ directly
 - supports richer feature set, generally better accuracy when trained over large supervised datasets
 - $P(t_i | w_i, t_{i-1}, x_i, y_i)$
 - E.g., Maximum Entropy Markov Model (MEMM), Conditional random field (CRF)
 - Most deep learning models of sequences are discriminative

HMMs in NLP

- HMMs are highly effective for part-of-speech tagging
 - ▶ trigram HMM gets 96.5% accuracy
 - ▶ related models are state of the art
 - ▶ MEMMs 97%
 - ▶ CRFs 97.6%
 - ▶ Deep CRF 97.9%
 - ▶ English Penn Treebank tagging accuracy [https://aclweb.org/aclwiki/index.php?title=POS_Tagging_\(State_of_the_art\)](https://aclweb.org/aclwiki/index.php?title=POS_Tagging_(State_of_the_art))
- Apply out-of-the box to other sequence labelling tasks
 - ▶ named entity recognition, shallow parsing, ...
 - ▶ In other fields: DNA, protein sequences...

A Final Word

- HMMs are a simple, yet effective way to perform sequence labelling.
- Can still be competitive, and fast. Natural baseline for other sequence labelling models.
- Main drawback: not very flexible in terms of feature representation, compared to MEMMs and CRFs.

Readings

- JM3 Chapter 17.4